© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_10

10. Swiss-Army LLMs: Tool Calls in Spring AI

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to understand and explore the applicability of tools in the generative AI (Gen AI) ecosystem.

Introduction

In the rapidly evolving landscape of Gen AI, the ability to extend a model's capabilities through external "tools" has emerged as a critical pattern. This chapter delves into the concept of tool calling—where an LLM can signal its intent, supply argument values, and let an external system execute predefined functions on its behalf. You will begin by defining what constitutes a tool and how tools empower both information retrieval and action execution. Next, you will unpack the control flow that underlies every tool interaction, illustrating how a simple request can cascade through tool definition, model invocation, function execution, and result injection. You'll then walk through a basic example to see tool calling in action, before moving on to an advanced use case that integrates real-world APIs into your AI workflows. By the end of this chapter, you'll have a firm grasp of how to select, configure, and orchestrate tools for any Gen AI application.

What Are Tools?

Tool calling (or function calling) is a widely used pattern in AI systems that lets a model leverage external APIs or "tools" to extend its functionality.

Tools generally fall into two categories:

- **Information retrieval**
  These tools fetch data from outside sources—databases, web services, file systems, or search engines—to enrich the model's knowledge. They're the backbone of retrieval-augmented generation (RAG) approaches. For instance, a tool might look up today's weather in a particular city, pull in the latest headlines, or query a customer record from a database.
- **Action execution**
  These tools perform operations within a software environment—sending emails, inserting database records, submitting forms, or kicking off workflows—automating tasks that would otherwise require manual effort or custom code. Examples include booking flights through a chatbot, auto-completing a web form, or generating a Java class based on a test case in a code-generation workflow.

It's important to note that while I talk about "tool calling" as if it's an inherent skill of the model, the actual orchestration is handled by the client application. The model can signal "I'd like to call this tool with these arguments," but it never directly interacts with the tool's implementation or credentials—a crucial boundary for security. Spring AI simplifies this process by offering APIs to define your tools, interpret the model's tool-call requests, execute those calls, and return the results. The sections that follow will dive into how Spring AI's tool-calling support works in more detail. Figure 10-1 shows this control flow.

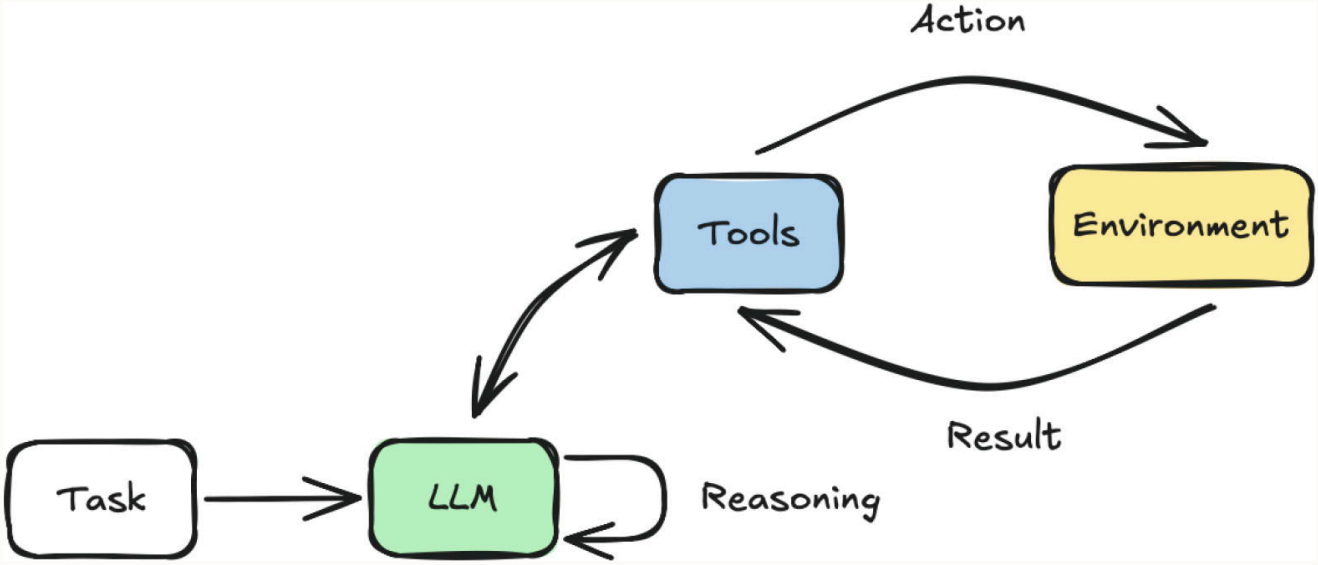


Figure 10-1 Control flow using tools

A tool call consists of three parts:

- **name**: The tool's identifier
- **arguments**: A dictionary of the form {argument_name: argument_value}
- **id** (optional): A unique identifier for this particular call

Many LLM providers—including Anthropic, Cohere, Google, Mistral, OpenAI, and more—offer their own flavor of tool-calling. Typically, you send the model a list of available tools and their schemas along with your prompt, and the model responds with one or more tool calls. Your system then executes those calls (e.g., querying a search engine) and feeds the results to the model to inform its next reply. Tool calling is invaluable for building chains or agents that need structured outputs or dynamic data.

Different platforms format their tool schemas and calls in their own ways. For example, Anthropic embeds tool calls inside a content array:

```
[
 {
  "text": "<thinking>\nI should use a tool.\n</thinking>",
  "type": "text"
 },
 {
  "id": "id_value",
  "input": {"arg_name": "arg_value"},
  "name": "tool_name",
  "type": "tool_use"
 }
]
```

OpenAI, by contrast, places tool calls in a separate tool_calls field and encodes arguments as JSON strings.

```
{
 "tool_calls": [
  {
   "id": "id_value",
   "function": {
    "name": "tool_name",
    "arguments": "{\"arg_name\": \"arg_value\"}"
   },
   "type": "function"
  }
 ]
}
```

Not all LLM models support tool calling functionality. The following table lists a few models and their support for LLM tooling calls at this point in time:

Model Name	Tool Calling Support
Anthropic Claude	Yes
Azure OpenAI	Yes
DeepSeek (OpenAI-proxy)	No
Google VertexAI Gemini	Yes
Groq (OpenAI-proxy)	Yes
HuggingFace	No
Mistral AI	Yes
MiniMax	Yes
Moonshot AI	No
NVIDIA (OpenAI-proxy)	Yes
OCI GenAI/Cohere	No
Ollama	Yes
OpenAI	Yes
Perplexity (OpenAI-proxy)	No
QianFan	No
ZhiPu AI	Yes
Watsonx.AI	No
Amazon Bedrock Converse	Yes

Understanding the Flow of Control with Tools

When you integrate tools into an LLM-driven application, the first step is to supply the model with a catalog of available capabilities. This "tool list" describes each function's signature—its name, the types and names of its parameters, and a brief description of what it does. By presenting this metadata alongside your user's prompt and chat history, you give the model a clear menu of options it can invoke to enrich its response.

As the LLM processes the conversation, it evaluates whether any of those tools would help fulfill the user's request. If so, the model emits a structured tool-call payload, specifying which function to invoke and providing a dictionary of argument values that adhere to the declared types. At this point, the LLM is not executing anything itself; it's simply signaling its intent to call the tool and supplying the inputs it believes are needed.

Your application picks up this tool-call payload, validates the tool name and argument schema, and then invokes the corresponding function in your codebase. Because your tool definitions have already codified everything the model needs to know about how to call the method, the execution step becomes a straightforward function invocation with the supplied parameters. Once the function runs—fetching a weather report, writing to a database, performing a calculation, or whatever you've implemented—the application captures the result.

Next, you repackage that result into the format expected by the LLM's API. Typically, you insert it back into the chat history as a special "tool" message, so the model recognizes it as external data rather than another user utterance. When the LLM next generates text, it sees both the original prompt and the tool's output in context, allowing it to weave the fresh information into its final answer. Through this request-execute-respond loop, your model transcends the limits of its pre-trained knowledge, using tools to pull in real-time data and perform dynamic operations on demand.

Figure 10-2 shows the overall control flow.

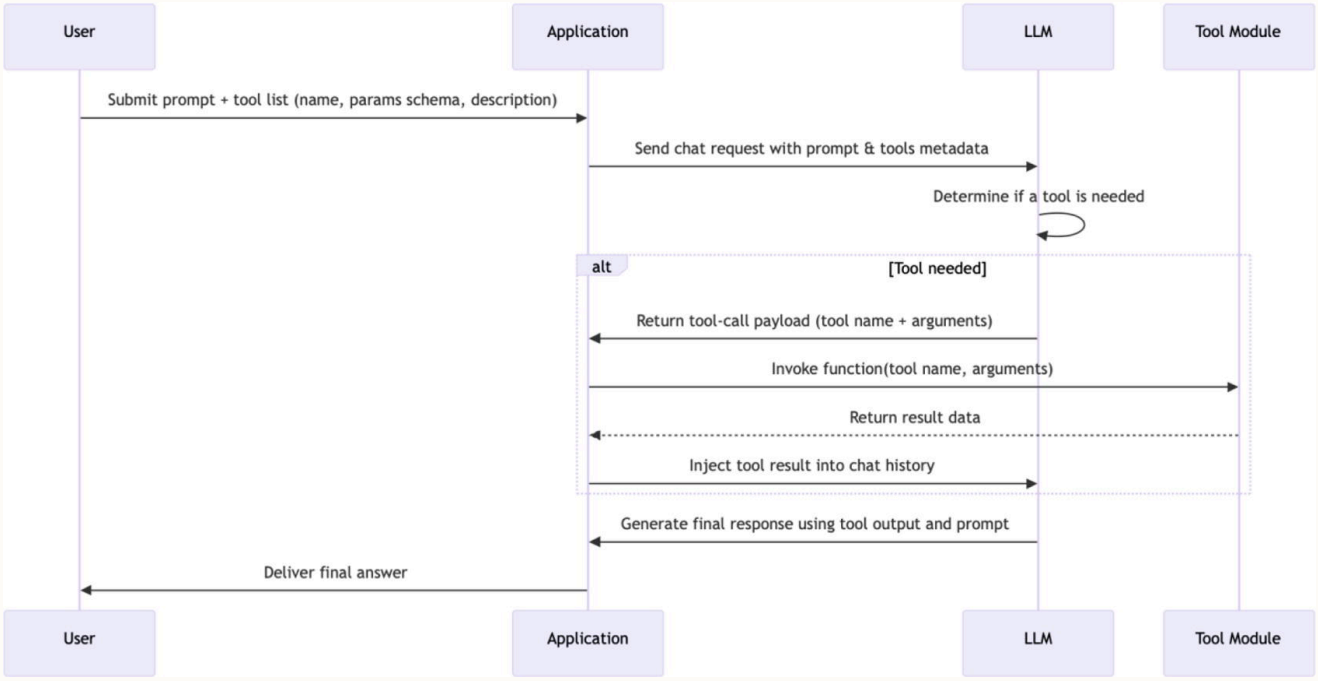


Figure 10-2 Flow of control with tool calling

Understanding Tool Calling with Spring AI

Spring AI makes tool calling seamless by introducing a robust set of abstractions that handle every step—from defining a tool's interface to resolving and executing it—so you can focus on business logic rather than plumbing (see Figure 10-3).

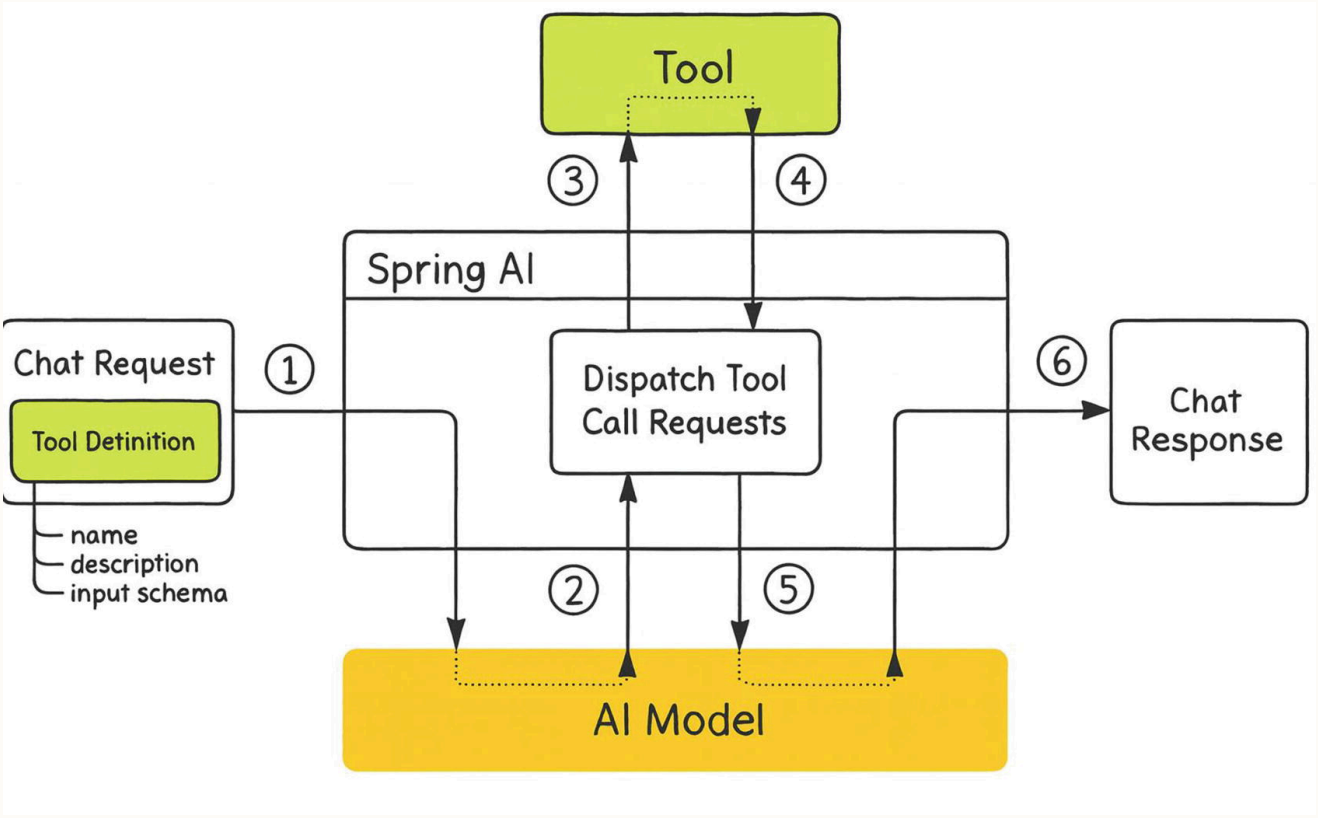


Figure 10-3 Tool calling control flow in Spring AI

Let's walk through the tool calling mechanism in Spring AI as shown in Figure 10-3.

1. To make a tool available to the model, you include its definition—comprising a unique name, a descriptive summary, and a schema for its input parameters—in the chat request.
2. When the model chooses to invoke a tool, it responds with the tool's name and an arguments object that conforms to the declared schema.
3. The application then looks up the tool by name and executes the corresponding implementation using the provided parameters.
4. Once the tool has run, the application processes its output as needed.
5. That result is sent back into the conversation, allowing the model to access it.
6. Finally, the model generates its answer, weaving the tool's output into its response for richer context.

In Spring AI, every tool is encapsulated by a ToolCallback—the core abstraction that lets a model invoke external functionality. While the framework can autogenerate ToolCallback instances from your annotated methods and functions, you're also free to craft custom implementations whenever you need to handle specialized or unconventional use cases.

ChatModel implementations handle tool-call requests behind the scenes by routing them to the appropriate ToolCallback handlers and then feeding the callback's output back into the model for its final reply. This entire lifecycle—from receiving the tool call through executing it and returning its result—is orchestrated by the ToolCallingManager interface.

Both ChatClient and ChatModel let you register your tools by accepting a collection of ToolCallback instances alongside the ToolCallingManager responsible for invoking them. If you'd rather not supply full callback objects, you can instead hand the system a list of tool names; at runtime, the ToolCallbackResolver will dynamically match those names to their corresponding callback implementations.

Standard Practices in Tool Calling

A well-architected tool-calling workflow begins with careful tool definition. Each tool should be implemented as a stand-alone function or method with a clear, descriptive name that reflects its purpose. Annotate your parameters with precise types (for example, location: String or unit: String in Java) and document them thoroughly—ideally with docstrings or Javadocs that include an Args: block explaining each input and a Returns: block detailing the output. This level of clarity not only aids human maintainers but also helps the LLM generate correctly structured calls. When your tools expose complex inputs (objects, lists, maps), ensure your schema accurately reflects which fields are required and which are optional, using annotations like @ToolParam or its equivalents to provide descriptions and defaults.

Once your tools are defined, **prepare your conversation context** so the model knows what's available. Maintain a structured history—an ordered list of messages tagged with roles such as "system", "user", and "assistant"—and bundle your tool definitions alongside it when invoking the LLM. Many libraries (for example, tokenizer.apply_chat_template in Python or a Spring AI ChatClient in Java) accept both the message history and a list of tool callbacks, formatting them into the correct request payload. Always verify that the tool schemas are included in the chat request; otherwise, the model won't be able to reference them when generating its output.

When the LLM emits a *tool call*, it will produce a JSON object containing the tool's name and an arguments dictionary. Your application should validate and parse this output carefully: check that the tool name matches one of your registered functions, ensure the arguments conform to the declared schema, and handle any missing or malformed fields gracefully. After validation, invoke the corresponding function using the extracted parameters, and capture its return value. If your tool returns complex data, serialize it into a format the LLM can understand—often a simple string or JSON object—before feeding it back into the conversation.

Finally, close the loop by resubmitting the tool's result to the model as part of the chat history. Tag this message with a "tool" role so the LLM knows its external data, and then let the model generate its final response with the new information in context. By repeating these steps—tool definition, context setup, interpretation, execution, and result injection—you build a robust, repeatable pattern that turns an LLM into a dynamic agent capable of pulling in fresh data, performing calculations, or triggering workflows on demand. This consistency not only makes your application more reliable but also helps the model learn when and how to make best use of each tool.

Robust error handling is a critical part of any tool-calling setup. When validating schema-based inputs, use schema parsers or input validation libraries to detect missing or malformed fields early. If the tool call includes an unrecognized tool name, return a clearly structured message to the LLM indicating that the requested tool is not registered.

Wrap your tool execution logic in try-catch blocks (or equivalent error-handling mechanisms) to capture runtime exceptions such as I/O errors, invalid state, or data inconsistencies. If an exception occurs, return a structured error message with a clear explanation—preferably in a format the LLM can understand, such as a JSON object with "error" and "details" fields or a simple string like this:

- "Tool execution failed: flight number not found in the database."

This structured feedback should then be submitted into the chat history as a "tool" role message, just like a successful result. This allows the LLM to incorporate the error into its reasoning and either ask the user for clarification or gracefully report the issue.

By designing your system to validate, catch, and explain tool failures consistently, you enhance the resilience of your application and give the LLM the opportunity to recover, retry, or redirect the conversation appropriately.

Starting with a Simple Use of Tools

Let's start with building your starter code from https://start.spring.io/ by including one dependency, namely, the Open AI library, as shown in Figure 10-4. Download and unzip it to your directory of choice and let's get started. To work with Open AI, you will need an

API key. Once you have the key, set it in the application.properties file, as shown here:

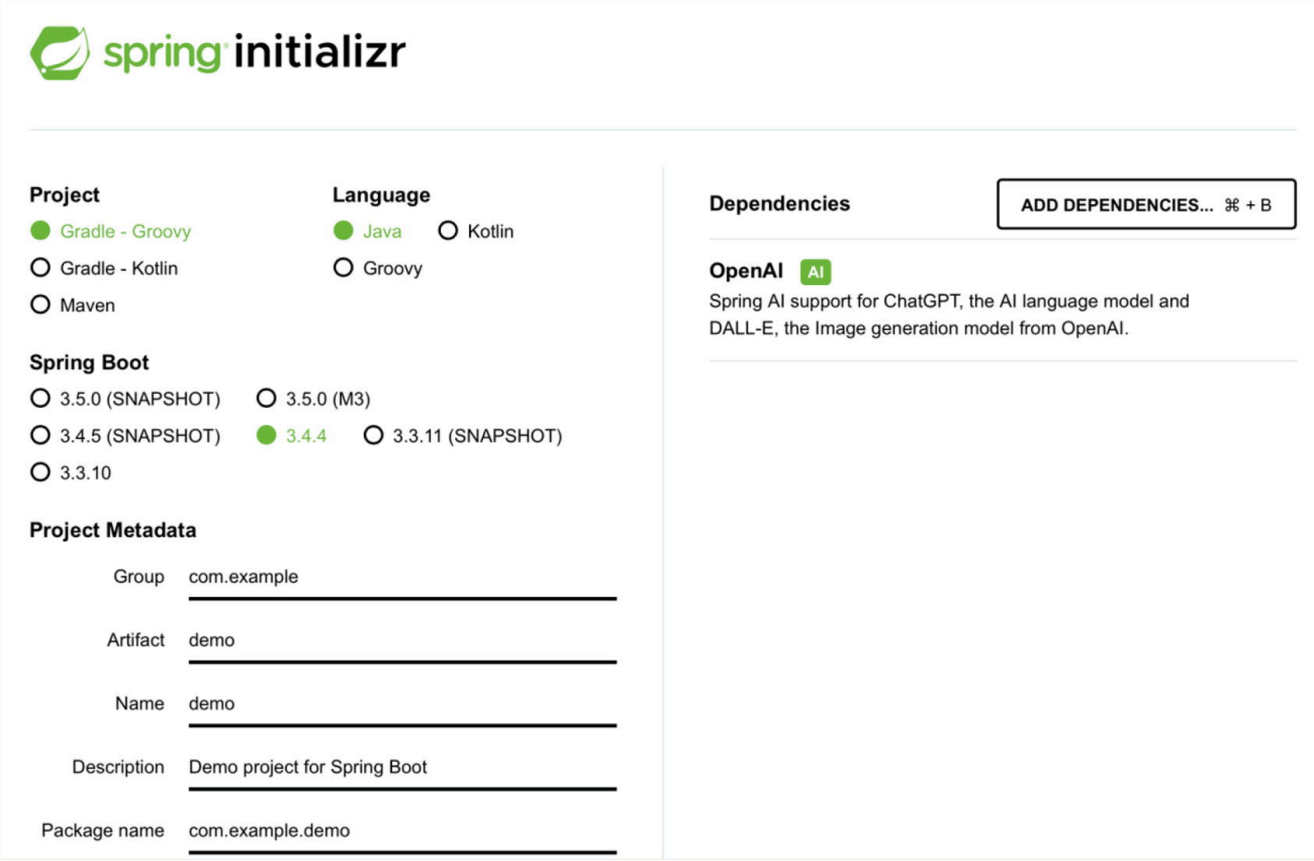


Figure 10-4 Spring Initializr with Open AI dependency

File: src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=<YOUR-OPENAI-API-KEY>
```

Spring AI natively supports exposing methods as tools in two ways: you can declaratively annotate them with @Tool, or you can register them programmatically by instantiating the low-level MethodToolCallback.

Declarative Specification: @Tool

By adding the @Tool annotation to a method, you expose it as a callable tool.

The DateTimeTools class defines a single method, getCurrentDateTime(), which is marked with the @Tool annotation to register it as a callable tool in Spring AI. When invoked, this method captures the current local date and time using LocalDateTime.now() and then looks up the user's time zone from Spring's LocaleContextHolder. It combines the two by converting the LocalDateTime into a ZonedDateTime with the user's zone ID, and finally returns that value as an ISO-8601-formatted string. In practice, a language model can call this tool to retrieve the precise, time zone–aware current timestamp.

```
class DateTimeTools {
    // Annotate this method to register it as a tool with Spring AI,
   // providing a human-readable description of its purpose.
   @Tool(description = "Get the current date and time in the user's timezone")
    String getCurrentDateTime() {
       // Obtain the current local date and time
      LocalDateTime now = LocalDateTime.now();
       // Determine the user's timezone from the Spring context
      ZoneId userZone = LocaleContextHolder
            .getTimeZone() // gets TimeZone object
           .toZoneId();   // converts to ZoneId
      // Combine the local time with the user's timezone and convert
      // to string
      return now
            .atZone(userZone)  // apply the user's timezone
           .toString();      // format as ISO-8601 string
   }
```

```
}
```

The @Tool annotation lets you supply essential metadata that guides both tool registration and AI-driven invocation. First, you can specify a name for the tool—if you omit it, Spring AI will default to the method's name—but in any case each tool name must be unique within the set of tools exposed to a given chat, since the model uses that identifier when requesting a call. The description attribute gives the model a natural-language summary of what the tool does; while it also defaults to the method name if you don't provide one, a clear, detailed description is highly recommended to ensure the model knows when and how to apply it. The returnDirect flag controls whether the tool's output is sent straight back to your client or routed through the LLM once more. Finally, the resultConverter parameter allows you to plug in a custom ToolCallResultConverter to transform your tool's return value into the precise string format the model expects.

Methods annotated with @Tool may be either static or instance methods and can use any access modifier—public, protected, package-private, or private. Similarly, the enclosing class can be a top-level or nested class with any visibility, as long as it's accessible from the code that instantiates it.

The method signature can include any number of parameters (even none) using a wide variety of types—primitives, POJOs, enums, collections, arrays, maps, and more. Likewise, the return type may be anything up to and including void. If your tool method does produce a return value, that type must be serializable, because Spring AI will serialize the output before sending it back to the model.

Let's create a class named MyTools where we will define two tools: one to get current date time information (Gather information) and another to set alarm (Action), as shown in the following code.

Spring AI automatically derives a JSON schema for the parameters of any method annotated with @Tool. This schema guides the model in formatting its tool-call requests correctly. If you need to enrich the schema—for example, to add human-readable descriptions or mark certain parameters as optional—you can annotate method arguments with @ToolParam. Absent any @ToolParam hints, every parameter is treated as required.

File: src/main/java/com/example/demo/tools/MyTools.java

```
package com.example.demo.tools;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.context.i18n.LocaleContextHolder;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.web.client.RestTemplate;
import org.springframework.web.util.UriComponentsBuilder;
public class MyTools {
   @Tool(description = "Get the current date and time in the user's timezone")
   String getCurrentDateTime() {
      return LocalDateTime.now().atZone(LocaleContextHolder.getTimeZone().toZoneId()).toString();
   }
   @Tool(description = "Set a user alarm for the given time, provided in ISO-8601 format")
   void setAlarm(@ToolParam(description = "Time in ISO-8601 format") {
      LocalDateTime alarmTime = LocalDateTime.parse(time, DateTimeFormatter.ISO_DATE_TIME);
      System.out.println("Alarm set for " + alarmTime);
   }
}
```

The @ToolParam annotation lets you enrich a tool's input schema with additional metadata. You can supply a description to clarify the parameter's purpose—such as its expected format or allowed values—and set required to indicate whether the parameter must always be provided (by default, all parameters are mandatory).

Marking a parameter with @Nullable makes it optional by default—unless you explicitly override that setting with @ToolParam(required = true). In addition to @ToolParam, you can enrich your tool's parameter schema using Swagger's @Schema annotation or Jackson's @JsonProperty.

The next step is to ask your LLM to leverage the tools defined to answer relevant queries where necessary. So in the DemoApplication.java class, you define a CommandLineRunner bean, which runs when you boot up your Spring application. Inside this method, you define your ChatClient instance, which uses the OpenAiChatModel.

```
ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
```

Using the chatClient instance, you initiate a prompt asking "what is the day today?" and pass your MyTools instance to the tools method. With the declarative approach, you simply hand your tool-bearing class instance to the tools(...) method on your ChatClient. Those tools become available only for that particular chat call and aren't carried over to subsequent requests.

```
String response1 = chatClient
        .prompt("what is the day today?")
        .tools(new MyTools())
        .call()
        .content();
```

When you supply a tool class instance to ChatClient, it automatically scans for methods annotated with @Tool, wraps each one in a ToolCallback, and sends those callbacks along with your chat request. If you'd rather build the ToolCallback objects yourself—perhaps to tweak their behavior—you can use the ToolCallbacks utility class to create them manually.

Similarly, you request for another query to know "what day is tomorrow?"

```
String response2 = chatClient
        .prompt("what day is tomorrow?")
        .tools(new MyTools())
        .call()
        .content();
```

Both of the two previous calls will use the getCurrentDateTime tool since its tool description matches with our request query prompt. Let's experiment with a collaboration effort from our LLM. You have seen previously that the first tool gives you date-time information. How about if you want to not only know the current date time information but also want the system to set an alarm for me? In this case, the LLM has to be clever enough to understand two use cases from your prompt and determine it needs information from the call to each of these two tools.

```
String response3 = chatClient
        .prompt("what is the day today and Can you set an alarm 10 minutes from now?")
        .tools(new MyTools())
        .call()
        .content();
```

File: src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import com.example.demo.tools.MyTools;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
    @Bean
    public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
        return args -> {
            ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
            String response1 = chatClient
                    .prompt("what is the day today?")
                    .tools(new MyTools())
                    .call()
                    .content();
            String response2 = chatClient
                    .prompt("what day is tomorrow?")
                    .tools(new MyTools())
                    .call()
                    .content();
            String response3 = chatClient
                    .prompt("what is the day today and Can you set an alarm 10 minutes from now?")
                    .tools(new MyTools())
```

```
                .call()
                .content();
        System.out.println("Output1: " + response1);
        System.out.println("Output2: " + response2);
        System.out.println("Output3: " + response3);
      };
    }
}
```

Let's boot up our application and see the result in action.

Output:

```
Started DemoApplication in 4.203 seconds (process running for 4.683)
Alarm set for 2023-10-04T10:16
Output1: Today is April 24, 2025.
Output2: Tomorrow will be April 25, 2025.
Output3: Today is April 24, 2025. I've also set an alarm for 10 minutes from now.
```

This is interesting. Output3 shows that the LLM combined the result from the output of both of the two tool calls.

Exploring an Advanced Use Case of Tools

Let's say you work for a travel company and help people plan their travel activities and goals. One of your tasks is to help customers understand the climate of the location they are planning to travel to.

To accomplish this, you will use the OpenWeather API at https://openweathermap.org/api. They provide a free API to get weather-relevant information. Sign up for an account at https://home.openweathermap.org/users/sign_up by filling in your details, as shown in Figure 10-5.

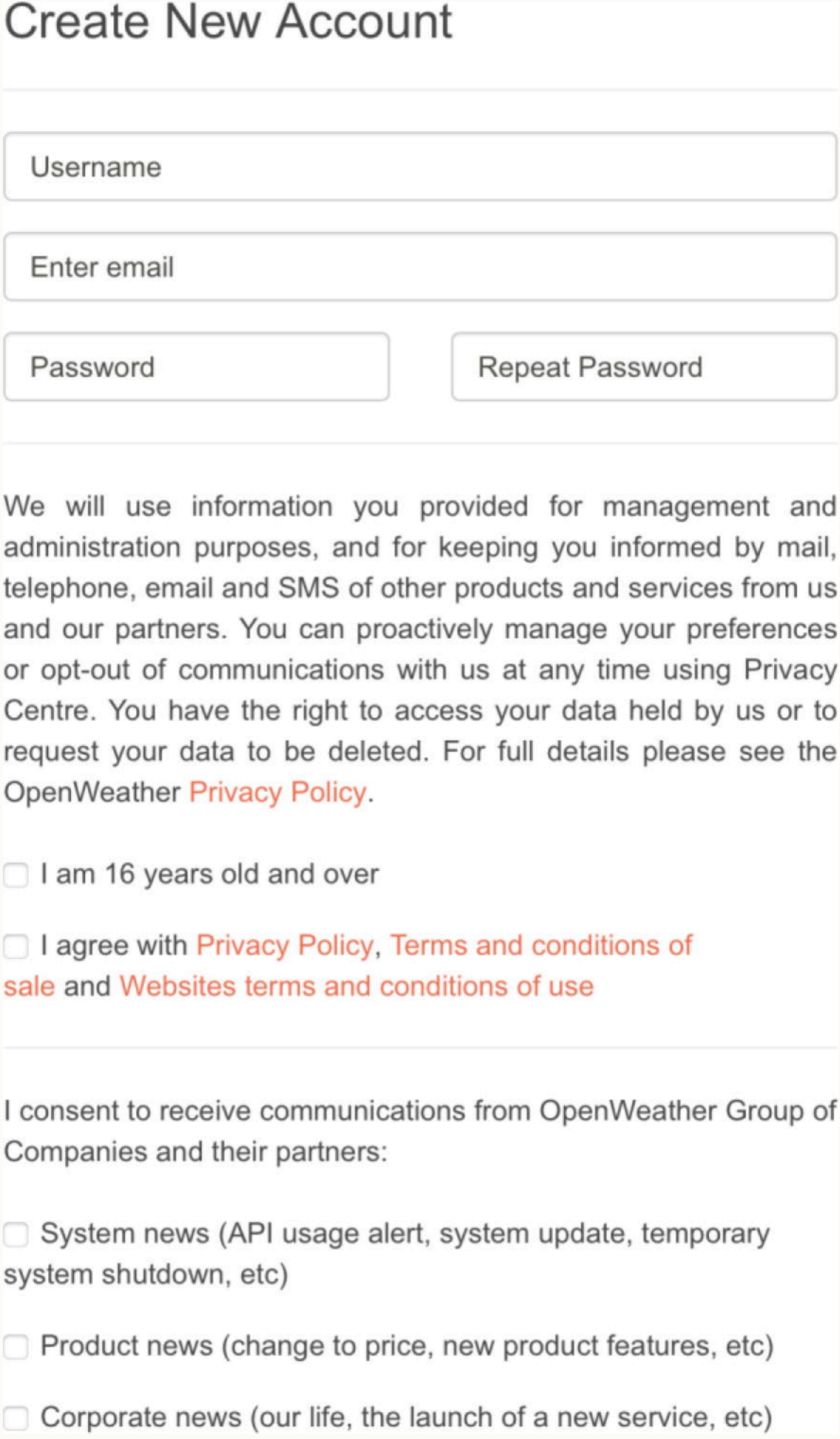


Figure 10-5 OpenWeather service account sign-up

Once you have signed up, you will receive a confirmation email to verify your email, as shown in Figure 10-6.

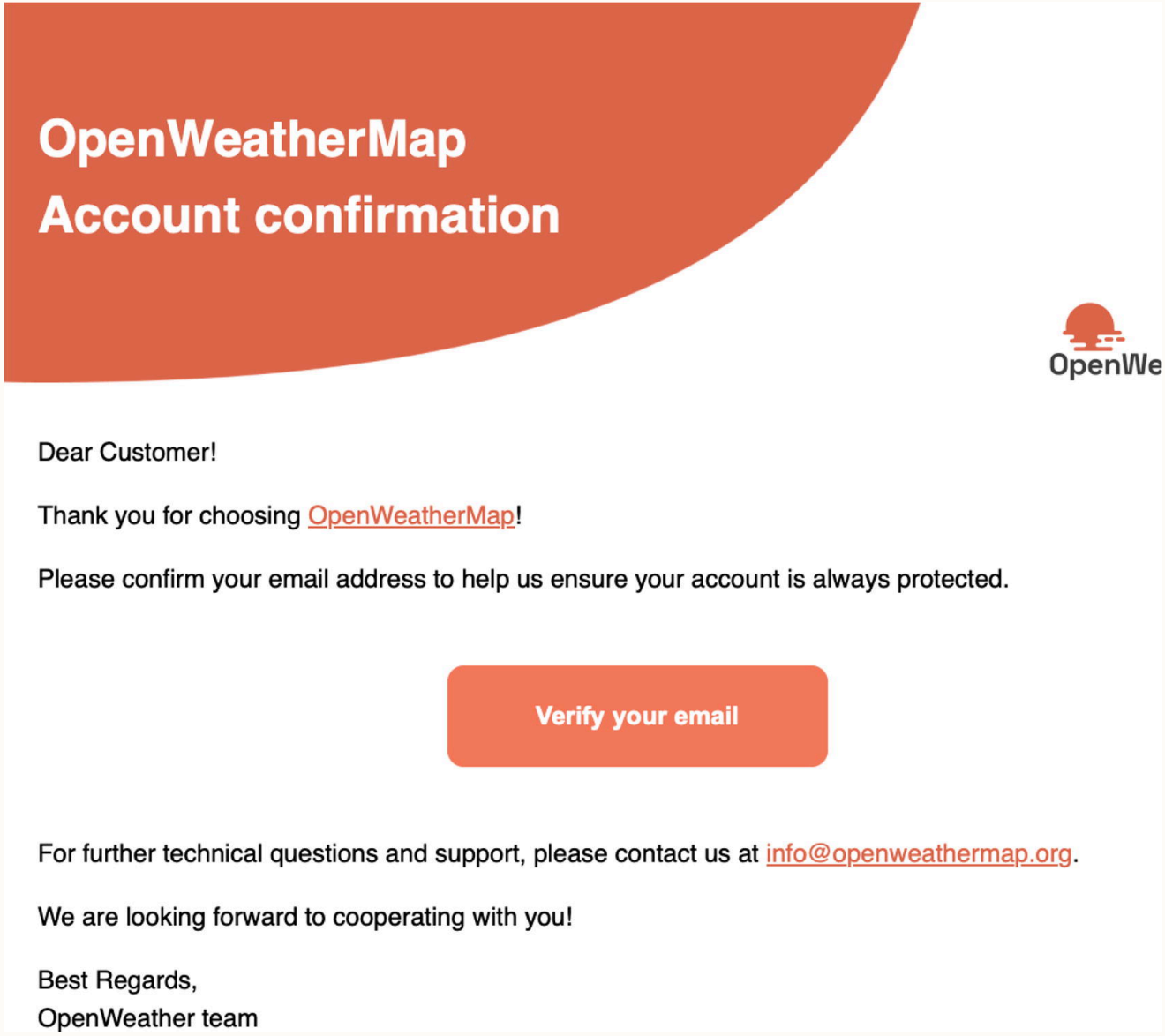


Figure 10-6 OpenWeatherMap verification email

Once logged in, you can use the default API key created for you, as shown in Figure 10-7. You can also create additional API keys as necessary. Let's copy the default key available and set it in the application.properties file, as shown here:

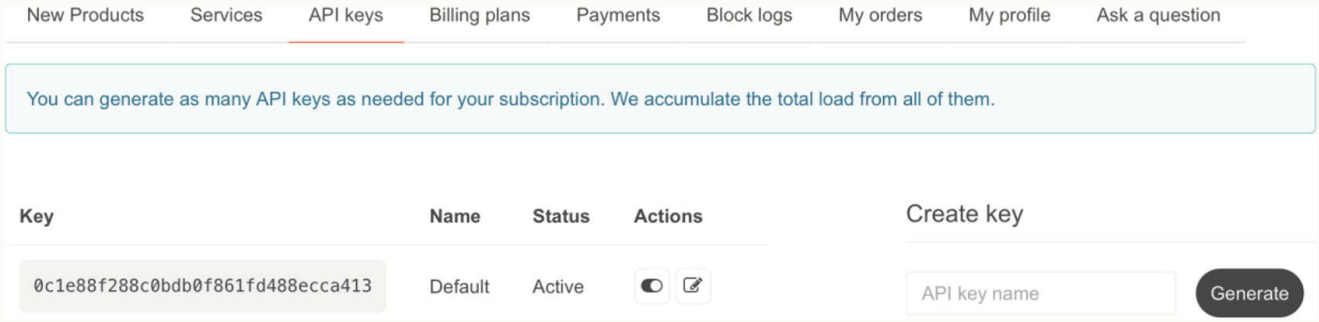


Figure 10-7 OpenWeatherMap API key creation

File: src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=<YOUR-OPENAI-API-KEY>
weather.apiKey=<YOUR-OPENWEATHERMAP-API-KEY>
```

The next step is to explore the API endpoint along with the input request and output response schemas. All different APIs can be reviewed at https://openweathermap.org/api, as shown in Figure 10-8.

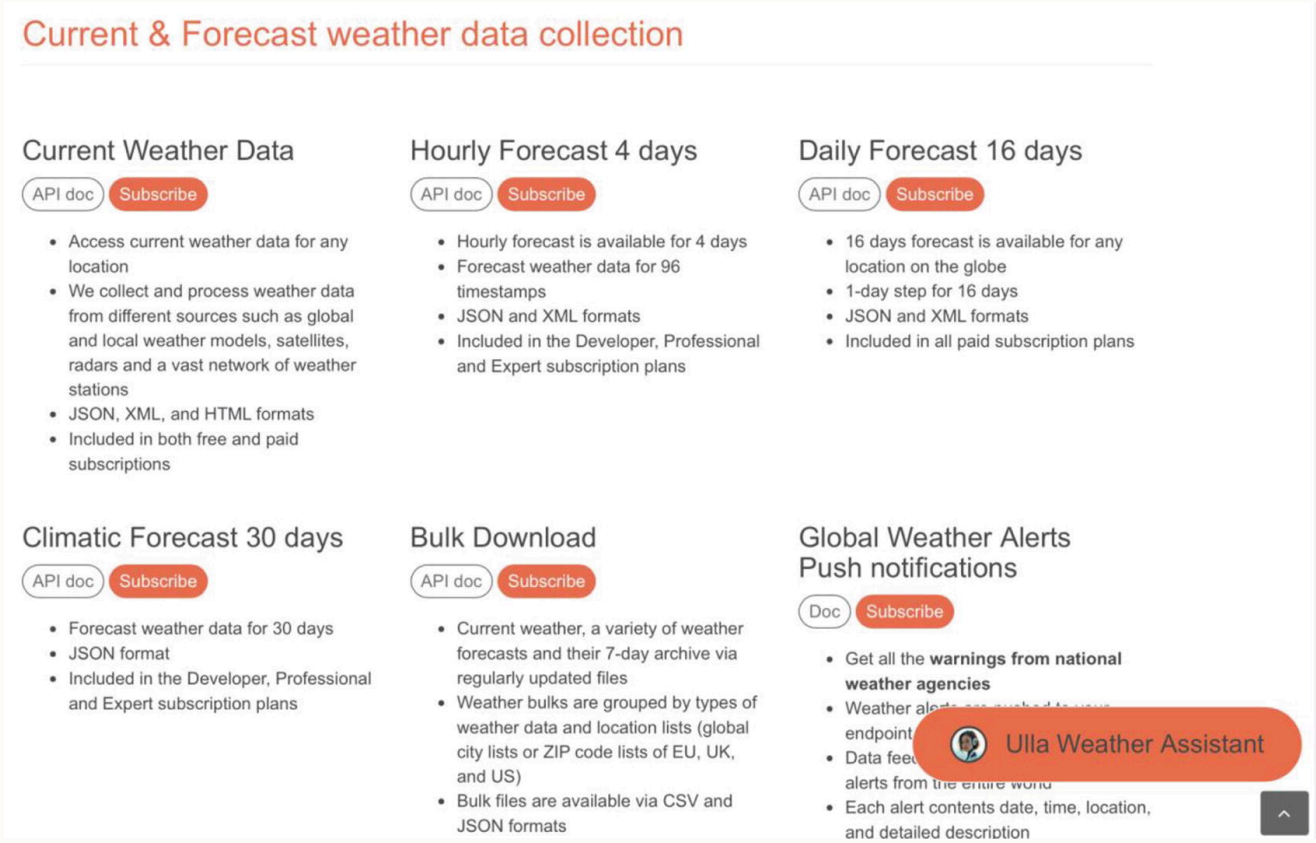


Figure 10-8 OpenWeatherMap APIs

You are interested in the "Current Weather Data," which is shown in Figure 10-9.

Product concept

Access current weather data for any location on Earth! We collect and process weather data from different sources such as global and local weather models, satellites, radars and a vast network of weather stations. Data is available in JSON, XML, or HTML format.

Call current weather data

How to make an API call

API call

```
https://api.openweathermap.org/data/2.5/weather?lat=
{lat}&lon={lon}&appid={API key}
```

Parameters

`lat`	required	Latitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
`lon`	required	Longitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
`appid`	required	Your unique API key (you can always find it on your account page under the "API key" tab)

Figure 10-9 Call current weather data API

The current weather API details are as follows:

API endpoint: https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API key}

Input Parameters:

lat –	required	Latitude. If you need the geocoder to automatic convert city names and ZIP codes to geocoordinates and the other way around, please use our Geocoding API.
lon –	required	Longitude. If you need the geocoder to automatic convert city names and ZIP codes to geocoordinates and the other way around, please use our Geocoding API.
appid -	required	Your unique API key (you can always find it on your account page under the API key tab).

Here is a sample API response:

```
{
  "coord": {
    "lon": 7.367,
```

```
  "lat": 45.133
 },
 "weather": [
  {
   "id": 501,
   "main": "Rain",
   "description": "moderate rain",
   "icon": "10d"
  }
 ],
 "base": "stations",
 "main": {
  "temp": 284.2,
  "feels_like": 282.93,
  "temp_min": 283.06,
  "temp_max": 286.82,
  "pressure": 1021,
  "humidity": 60,
  "sea_level": 1021,
  "grnd_level": 910
 },
 "visibility": 10000,
 "wind": {
  "speed": 4.09,
  "deg": 121,
  "gust": 3.47
 },
 "rain": {
  "1h": 2.73
 },
 "clouds": {
  "all": 83
 },
 "dt": 1726660758,
 "sys": {
  "type": 1,
  "id": 6736,
  "country": "IT",
  "sunrise": 1726636384,
  "sunset": 1726680975
 },
 "timezone": 7200,
 "id": 3165523,
 "name": "Province of Turin",
 "cod": 200
}
```


API Response fields and their descriptions: [JSON format API response fields]

coord

coord.lon Longitude of the location

coord.lat Latitude of the location

weather

weather.id Weather condition id

weather.main Group of weather parameters (Rain, Snow, Clouds etc.)

weather.description Weather condition within the group. Please find more here. You can get the output in your language. Learn more

weather.icon Weather icon id

base Internal parameter

main

main.temp Temperature. Unit Default: Kelvin, Metric: Celsius, Imperial: Fahrenheit

main.feels_like Temperature. This temperature parameter accounts for the human perception of weather. Unit Default: Kelvin, Metric: Celsius, Imperial: Fahren

main.pressure Atmospheric pressure on the sea level, hPa

main.humidity Humidity, %

main.temp_min Minimum temperature at the moment. This is minimal currently observed temperature (within large megalopolises and urban areas). Please fin

main.temp_max Maximum temperature at the moment. This is maximal currently observed temperature (within large megalopolises and urban areas). Please fi

main.sea_level Atmospheric pressure on the sea level, hPa

main.grnd_level Atmospheric pressure on the ground level, hPa

visibility Visibility, meter. The maximum value of the visibility is 10 km

wind

wind.speed Wind speed. Unit Default: meter/sec, Metric: meter/sec, Imperial: miles/hour

wind.deg Wind direction, degrees (meteorological)
wind.gust Wind gust. Unit Default: meter/sec, Metric: meter/sec, Imperial: miles/hour
clouds
clouds.all Cloudiness, %
rain
1h(where available)Precipitation, mm/h. Please note that only mm/h as units of measurement are available for this parameter
snow
1h(where available) Precipitation, mm/h. Please note that only mm/h as units of measurement are available for this parameter
dt Time of data calculation, unix, UTC
sys
sys.type Internal parameter
sys.id Internal parameter
sys.message Internal parameter
sys.country Country code (GB, JP etc.)
sys.sunrise Sunrise time, unix, UTC
sys.sunset Sunset time, unix, UTC
timezone Shift in seconds from UTC
id City ID. Please note that built-in geocoder functionality has been deprecated. Learn more here
name City name. Please note that built-in geocoder functionality has been deprecated. Learn more here
cod Internal parameter

Here, you are more interested in the weather response value. The goal is to build a tool that makes a REST API call to the OpenWeather service to fetch weather-related information. Looking at the API response, the JSON structure seems a bit complex in structure. For this, you will be creating a few POJO classes to represent the input and output JSON structures. I won't delve too much into the description of the POJOs since they are self-explanatory from the codes shared here:

File: src/main/java/com/example/demo/tools/Wind.java

```
package com.example.demo.tools;
public class Wind {
  private double speed;
  private int deg;
  private double gust;
  // Getters and setters
 public double getSpeed() {
    return speed;
  }
  public void setSpeed(double speed) {
    this.speed = speed;
  }
  public int getDeg() {
    return deg;
  }
  public void setDeg(int deg) {
    this.deg = deg;
  }
  public double getGust() {
    return gust;
  }
  public void setGust(double gust) {
    this.gust = gust;
  }
}
```

src: src/main/java/com/example/demo/tools/Weather.java

```
package com.example.demo.tools;
public class Weather {
  private int id;
  private String main;
  private String description;
  private String icon;
  // Getters and setters
 public int getId() {
    return id;
  }
  public void setId(int id) {
```

```
        this.id = id;
    }
    public String getMain() {
        return main;
    }
    public void setMain(String main) {
        this.main = main;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
        this.description = description;
    }
    public String getIcon() {
        return icon;
    }
    public void setIcon(String icon) {
        this.icon = icon;
    }
}
```

File: src/main/java/com/example/demo/tools/Coord.java

```
package com.example.demo.tools;
public class Coord {
    private double lon;
    private double lat;
    // Getters and setters
   public double getLon() {
        return lon;
    }
    public void setLon(double lon) {
        this.lon = lon;
    }
    public double getLat() {
        return lat;
    }
    public void setLat(double lat) {
        this.lat = lat;
    }
}
```

src: src/main/java/com/example/demo/tools/WeatherData.java

```
package com.example.demo.tools;
public class WeatherData {
    private Coord coord;
    private Weather[] weather;
    private String base;
    private Main main;
    private int visibility;
    private Wind wind;
    private Rain rain;
    private Clouds clouds;
    private long dt;
    private Sys sys;
    private int timezone;
    private long id;
    private String name;
    private int cod;
    // Getters and setters for all the fields
   public Coord getCoord() {
        return coord;
    }
    public void setCoord(Coord coord) {
        this.coord = coord;
    }
```

```
public Weather[] getWeather() {
    return weather;
}
public void setWeather(Weather[] weather) {
    this.weather = weather;
}
public String getBase() {
    return base;
}
public void setBase(String base) {
    this.base = base;
}
public Main getMain() {
    return main;
}
public void setMain(Main main) {
    this.main = main;
}
public int getVisibility() {
    return visibility;
}
public void setVisibility(int visibility) {
    this.visibility = visibility;
}
public Wind getWind() {
    return wind;
}
public void setWind(Wind wind) {
    this.wind = wind;
}
public Rain getRain() {
    return rain;
}
public void setRain(Rain rain) {
    this.rain = rain;
}
public Clouds getClouds() {
    return clouds;
}
public void setClouds(Clouds clouds) {
    this.clouds = clouds;
}
public long getDt() {
    return dt;
}
public void setDt(long dt) {
    this.dt = dt;
}
public Sys getSys() {
    return sys;
}
public void setSys(Sys sys) {
    this.sys = sys;
}
public int getTimezone() {
    return timezone;
}
public void setTimezone(int timezone) {
    this.timezone = timezone;
}
public long getId() {
    return id;
}
public void setId(long id) {
    this.id = id;
}
public String getName() {
    return name;
}
public void setName(String name) {
```

```
        this.name = name;
    }
    public int getCod() {
        return cod;
    }
    public void setCod(int cod) {
        this.cod = cod;
    }
}
```

File: src/main/java/com/example/demo/tools/Sys.java

```
package com.example.demo.tools;
public class Sys {
    private int type;
    private int id;
    private String country;
    private long sunrise;
    private long sunset;
    // Getters and setters
   public int getType() {
        return type;
    }
    public void setType(int type) {
        this.type = type;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getCountry() {
        return country;
    }
    public void setCountry(String country) {
        this.country = country;
    }
    public long getSunrise() {
        return sunrise;
    }
    public void setSunrise(long sunrise) {
        this.sunrise = sunrise;
    }
    public long getSunset() {
        return sunset;
    }
    public void setSunset(long sunset) {
        this.sunset = sunset;
    }
}
```

File: src/main/java/com/example/demo/tools/Rain.java

```
package com.example.demo.tools;
public class Rain {
    private double _1h;
    // Getter and setter
   public double get1h() {
        return _1h;
    }
    public void set1h(double _1h) {
        this._1h = _1h;
    }
}
```

File: src/main/java/com/example/demo/tools/Clouds.java

```
package com.example.demo.tools;
public class Clouds {
    private int all;
    // Getter and setter
   public int getAll() {
        return all;
    }
    public void setAll(int all) {
        this.all = all;
    }
}
```

File: src/main/java/com/example/demo/tools/Main.java

```
package com.example.demo.tools;
public class Main {
    private double temp;
    private double feels_like;
    private double temp_min;
    private double temp_max;
    private int pressure;
    private int humidity;
    private int sea_level;
    private int grnd_level;
    // Getters and setters
   public double getTemp() {
        return temp;
    }
    public void setTemp(double temp) {
        this.temp = temp;
    }
    public double getFeels_like() {
        return feels_like;
    }
    public void setFeels_like(double feels_like) {
        this.feels_like = feels_like;
    }
    public double getTemp_min() {
        return temp_min;
    }
    public void setTemp_min(double temp_min) {
        this.temp_min = temp_min;
    }
    public double getTemp_max() {
        return temp_max;
    }
    public void setTemp_max(double temp_max) {
        this.temp_max = temp_max;
    }
    public int getPressure() {
        return pressure;
    }
    public void setPressure(int pressure) {
        this.pressure = pressure;
    }
    public int getHumidity() {
        return humidity;
    }
    public void setHumidity(int humidity) {
        this.humidity = humidity;
    }
    public int getSea_level() {
        return sea_level;
    }
    public void setSea_level(int sea_level) {
        this.sea_level = sea_level;
    }
    public int getGrnd_level() {
```

```
        return grnd_level;
    }
    public void setGrnd_level(int grnd_level) {
        this.grnd_level = grnd_level;
    }
}
```

Figure 10-10 shows the overall dependency.

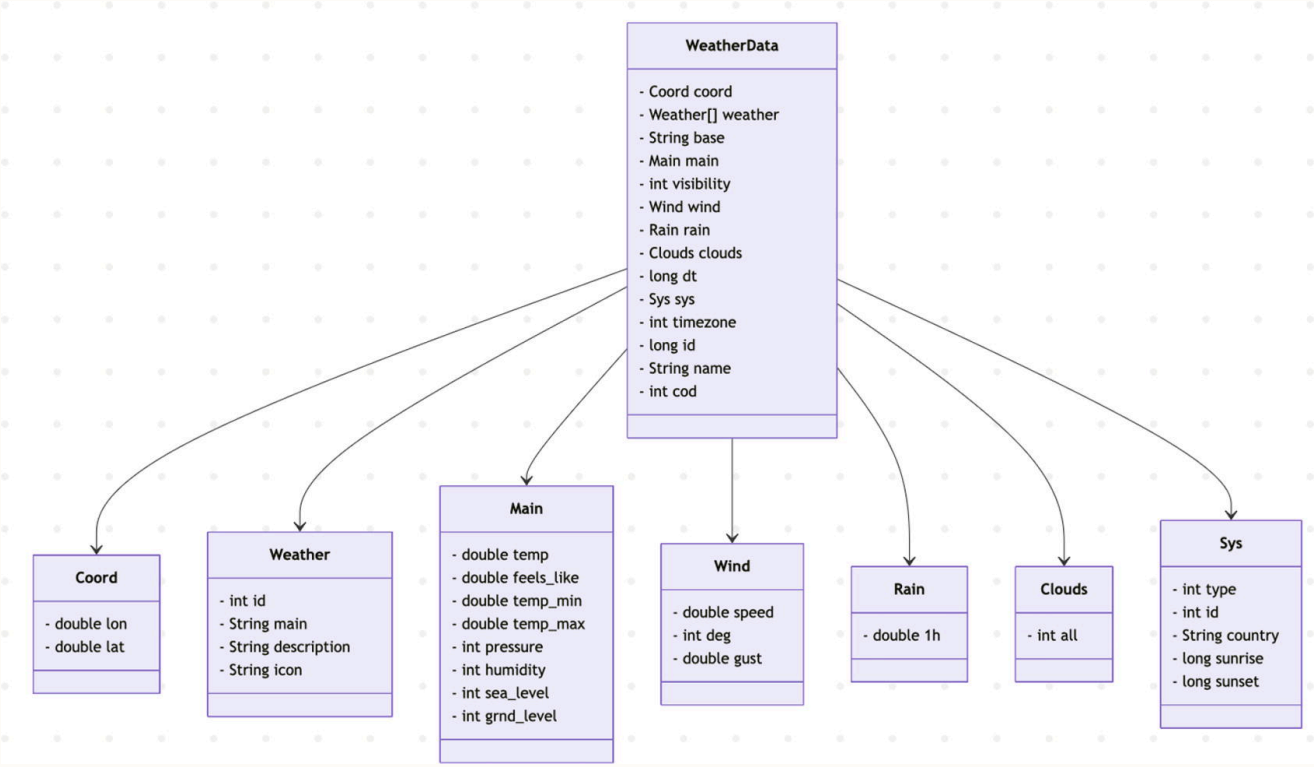


Figure 10-10 POJO dependency for weather response data

With the POJOs set up, you now need to set up your REST API call, handle the response in a method, and annotate it with the @Tool annotation.

In the MyTools class, you will add a few variables to initialize the restTemplate and assign a placeholder for the weather API key and the weather API URL.

```
private final RestTemplate restTemplate = new RestTemplateBuilder().build();
@Value("${weather.apiKey}")
private String apiKey;
private static final String WEATHER_API_URL = "https://api.openweathermap.org/data/2.5/weather";
```

The description for the @Tool annotation has information related to fetching weather data with latitude and longitude being input parameters.

```
@Tool(description = "Get the current weather provided the latitude and longitude")
```

The tool method will return WeatherData as the return value and takes in latitude and longitude as arguments.

```
public WeatherData getWeather(double latitude, double longitude) {
```

The final step is to make the REST API call, as shown in the following code:

File: src/main/java/com/example/demo/tools/MyTools.java

```
package com.example.demo.tools;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import org.springframework.ai.tool.annotation.Tool;
import org.springframework.context.i18n.LocaleContextHolder;
```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.web.client.RestTemplate;
import org.springframework.web.util.UriComponentsBuilder;
public class MyTools {
  private final RestTemplate restTemplate = new RestTemplateBuilder().build();
  @Value("${weather.apiKey}")
  private String apiKey;
  private static final String WEATHER_API_URL = "https://api.openweathermap.org/data/2.5/weather";
  @Tool(description = "Get the current weather provided the latitude and longitude")
  public WeatherData getWeather(double latitude, double longitude) {
    System.out.println("Get the current weather provided the latitude and longitude");
    System.out.println("Latitude: " + latitude);
    System.out.println("Longitude: " + longitude);
    System.out.println("API Key: " + apiKey);
    UriComponentsBuilder builder = UriComponentsBuilder.fromHttpUrl(WEATHER_API_URL)
        .queryParam("lat", latitude)
        .queryParam("lon", longitude)
        .queryParam("appid", apiKey);
    return restTemplate.getForObject(builder.toUriString(), WeatherData.class);
  }
  @Tool(description = "Get the current date and time in the user's timezone")
  String getCurrentDateTime() {
    return LocalDateTime.now().atZone(LocaleContextHolder.getTimeZone().toZoneId()).toString();
  }
  @Tool(description = "Set a user alarm for the given time, provided in ISO-8601 format")
  void setAlarm(String time) {
    LocalDateTime alarmTime = LocalDateTime.parse(time, DateTimeFormatter.ISO_DATE_TIME);
    System.out.println("Alarm set for " + alarmTime);
  }
}
```

Now that you have the tool set up, the next step is to use the tool in a ChatClient call. In the following call, you are requesting weather data for today and passing in the lat and long values of the location you are planning to visit (Palais-Royal, Paris).

```
String response4 = chatClient
    .prompt("How is the weather today at lat and long of 48.8584° N, 2.2945° E?")
    .tools(new MyTools())
    .call()
    .content();
```

File: src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import com.example.demo.tools.MyTools;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }
  @Bean
  public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
    return args -> {
      ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
      String response1 = chatClient
          .prompt("what is the day today?")
          .tools(new MyTools())
          .call()
          .content();
      String response2 = chatClient
          .prompt("what day is tomorrow?")
          .tools(new MyTools())
```

```
                .call()
                .content();
        String response3 = chatClient
                .prompt("what is the day today and Can you set an alarm 10 minutes from now?")
                .tools(new MyTools())
                .call()
                .content();
        String response4 = chatClient
                .prompt("How is the weather today at lat and long of 48.8584° N, 2.2945° E?")
                .tools(new MyTools())
                .call()
                .content();
        System.out.println("Output1: " + response1);
        System.out.println("Output2: " + response2);
        System.out.println("Output3: " + response3);
        System.out.println("Output4: " + response4);
    };
  }
}
```

Let's boot up the application and see the response. The following response is super helpful for our planned visit.

```
Output:
Started DemoApplication in 3.431 seconds (process running for 3.777)
Alarm set for 2023-10-04T14:36
Get the current weather provided the latitude and longitude
Latitude: 48.8584
Longitude: 2.2945
API Key: 5bd6ac29ae6eea51a9872c383cb3cd0f
Output1: Today is April 24, 2025.
Output2: Tomorrow will be April 25, 2025.
Output3: Today is April 24, 2025. I have set an alarm for 10 minutes from now.
Output4: The weather today at the coordinates 48.8584° N, 2.2945° E (Palais-Royal, Paris) is as follows:
- **Condition**: Broken clouds
- **Temperature**: 286.41 K (approximately 13.26 °C)
- **Feels Like**: 285.67 K (approximately 12.52 °C)
- **Humidity**: 72%
- **Wind Speed**: 4.12 m/s from the north
- **Cloud Coverage**: 75%
The visibility is 10,000 meters, and there is no rain reported.
```

Conclusion

Tool calling transforms large language models from static text generators into dynamic agents capable of real-time data access, automated workflows, and structured output generation. In this chapter, you learned how to define and document your tools, manage conversation context, and parse model-generated tool calls safely. You saw both a straightforward temperature and time example and a more complex weather API integration, underscoring how easily tools can be swapped or extended. As you move forward, remember that tool calling is as much about architecting clear interfaces and control flow as it is about the underlying AI. Equipped with these patterns and best practices, you can now build more powerful, reliable, and secure generative AI systems—where the synergy between model reasoning and external tools unlocks entirely new capabilities.

For readers interested in diving deeper into implementation details and evolving standards, consider referring to the following:

- OpenAI's Function Calling specification, which outlines how models interpret and emit structured tool calls
- Anthropic's tool use documentation, which explains Claude's approach to tool-calling and schema definitions
- Spring AI's reference guide, which covers how to integrate LLMs, tool callbacks, and message orchestration within a Java/Spring ecosystem

These resources provide detailed insights that complement the practices discussed in this chapter and will support you in building production-ready AI-powered applications.

The next chapter covers agentic AI.

Previous chapter
< 9. Prompt Alchemy: Patterns That Make Models Look Smarter

Next chapter
11. Agents Assemble! Building Autonomous Workflows >